

7. AI 기반의 체계적인 개발과 관리



1) 컨텍스트 윈도우 관리



컨텍스트 윈도우란?

- 컨텍스트 윈도우(Context Window)는 인공지능(AI) 대형 언어 모델이 한 번에 기억하고 처리할 수 있는 최대 정보량의 단위입니다.
 - 사용자의 질문, 이전 대화 내용, AI의 답변, 그리고 시스템 지시사항이 모두 이 공간 안에 포함됩니다.
- 컨텍스트 윈도우의 한계
 - 실습 중 대화가 길어지면 AI가 앞서 준 지시나 규칙을 잊어버리는 경험, 한 번쯤 있었을 것입니다.
 - 이는 AI 모델이 한 번에 인식·처리할 수 있는 텍스트 범위, 즉 '컨텍스트 윈도우'에 한계가 있기 때문입니다.
 - 이 범위를 벗어난 정보는 AI가 인식하지 못하거나 잘려서 무시될 수 있습니다.
 - 처리 가능한 크기는 모델마다 다르며, 기술 발전에 따라 계속 커지고 있습니다.

컨텍스트 윈도우의 핵심구성 요소

■ 토큰

- AI는 텍스트를 '토큰'이라는 단위로 잘게 쪼개어 처리합니다.
- 영어는 단어 또는 단어의 일부 단위로, 한글은 상대적으로 더 잘게 쪼개지는 경향이 있습니다.
- 컨텍스트 윈도우의 크기는 바로 이 토큰의 개수로 표현됩니다.

■ 입력 토큰: 시스템 지시사항, 사용자 질문, 이전 대화 기록입니다.

■ 출력 토큰: 인공지능이 답변으로 생성해 내는 텍스트를 의미합니다.

■ 토큰 단위: 글자와 단어가 조합된 처리 단위

컨텍스트 윈도우 주요 특징 및 한계

■ 용량 제한:

- 모델에 따라 수천 개에서 수백만 개의 토큰을 지원하지만 용량제한으로 프로젝트의 코드나 문서 전체를 한 번에 모두 넣을 수 없는 경우가 많습니다.

■ 기억의 한계:

- 대화가 길어지면 앞부분의 지시나 맥락을 AI가 더 이상 참조하지 못할 수 있습니다.
- 윈도우 범위를 벗어난 오래된 정보는 AI가 완전히 잊어버리게 됩니다.

■ 품질 저하:

- 한계에 가까워질수록 앞선 내용을 누락하거나 지시를 놓치는 현상이 발생할 수 있습니다
- 불필요한 정보가 많으면 정작 중요한 규칙이 뒤로 밀려 AI가 놓칠 수 있습니다.

컨텍스트 윈도우를 효율적으로 쓰는 전략

- 핵심 정보만 간결하게 정리해 우선순위를 명확히 합니다.
- 작업에 필요한 파일·문서만 선택적으로 포함합니다.
- 대화가 길어지면 중간에 요약하거나, 새 세션에서 핵심 맥락만 다시 제공합니다.
- 'MD 문서 작성'은 컨텍스트를 효율적으로 관리하는 실질적인 도구가 됩니다.

2) MD 문서로 컨텍스트 관리하기



왜 문서가 필요한가? – AI 협업의 컨텍스트 문제

- 생성형 AI는 대화가 길어지거나 새 세션이 시작되면 이전 맥락을 온전히 기억하지 못합니다.
- 매번 프로젝트 배경과 규칙을 새로 설명해야 한다면 반복 작업이 늘고 결과의 일관성도 떨어집니다.
- 문서는 다음과 같은 역할을 합니다
 - AI와 사람이 함께 참조하는 공유된 기준점(Single Source of Truth) 제공
 - 매 대화마다 반복 설명해야 하는 정보를 줄여 효율성 확보
 - 여러 명이 협업할 때도 동일한 맥락과 규칙을 유지

MD(마크다운) 파일이란

- 제목, 목록, 표, 코드 등을 간단한 기호로 표현하는 경량 텍스트 서식입니다.
 - #, -, 1., ``` 같은 기호로 문서의 구조와 위계를 명확히 구분합니다.
 - 특별한 프로그램 없이 일반 텍스트 편집기로 작성·열람이 가능합니다.
 - 코드 저장소(Git)에서 코드와 함께 버전 관리를 할 수 있습니다.
- 사람이 읽기 쉬우면서 기계(AI)가 구조를 해석하기도 쉬워, AI 협업 문서의 표준 형식으로 널리 쓰입니다.

체계적인 MD 문서 = 신뢰할 수 있는 AI 컨텍스트

MD 문서 = 맥락과 규칙을 담은 기준 문서

AI와 사람이 함께 참조하는 자료

프로젝트 목적과 범위 명시

코딩 규칙과 제약 조건 전달

현재 상태와 작업 이력 공유

=

반복 설명 없이 일관된 결과

동일한 기준으로 협업

AI 응답 근거 검증 가능

MD 문서파일로 AI 개발 효율 UP

- MD 문서 파일은 AI의 "기억 시스템"

- 프로젝트 규칙을 작성해 두어서 AI가 사용자의 선호도를 기억하고, 생성되는 코드, 문서, PR 설명문까지도 프로젝트 스타일에 맞게 작성하는 Markdown 형식의 파일입니다.
- AI에게 프로젝트를 반복해서 설명할 필요 없이, 그 시간을 코드 작성이나 로직 설계에 집중할 수 있습니다.

- 주의사항

- API 키나 비밀번호와 같은 민감한 정보는 이 파일에 기록하지 말고 환경 변수나 시크릿 관리를 사용하는 것이 안전합니다.

AI를 위한 "프로젝트 설명서"

- MD 파일로 AI에게 전달할 수 있는 내용:

- 프로젝트의 코딩 규약과 모범 사례
- 코드 스타일 선호도
- 자주 사용하는 라이브러리와 프레임워크
- 프로젝트 배경과 비즈니스 로직

- 예시:

- React + TypeScript 프로젝트에서 사용할 경우, 컴포넌트 명명 규칙, 상태 관리 방법, 테스트 커버리지 요구사항 등을 모두 작성해 두면 AI가 생성하는 코드는 거의 수정이 필요 없게 됩니다.

프로젝트 문서 유형과 목적

- 프로젝트 성격에 맞춰 아래 문서를 상황에 따라 선택적으로 구성합니다.

문서 유형	목적	포함 내용
README	프로젝트 개요 파악	프로젝트 소개, 설치·실행 방법, 폴더 구조
PRD (요구사항 정의서)	무엇을 만들지 정의	목표, 기능 요구사항, 범위(포함/제외)
코딩 규칙·컨벤션 문서	AI와 사람이 함께 지킬 규칙 전달	코드 스타일, 명명 규칙, 금지 사항, 기술 스택
아키텍처·설계 문서	구조와 의사결정 배경 공유	시스템 구성도, 주요 설계 결정과 이유
체인지로그(변경 이력)	무엇이, 왜 바뀌었는지 추적	버전별 변경 사항, 작업 로그

좋은 MD 문서 작성 원칙

- AI와 사람이 오해 없이 활용할 수 있으려면 다음 원칙을 지켜 작성합니다.
 - 명확성: 모호한 표현 대신 구체적인 규칙과 기준으로 서술합니다.
 - 구조화: 제목 계층(#, ##)과 목록을 활용해 정보를 논리적으로 나눕니다.
 - 최신성: 코드와 정책이 바뀌면 문서도 함께 갱신해 '문서 부채'를 방지합니다.
 - 최소 필요성: 꼭 필요한 정보만 담아 AI가 참고할 컨텍스트 양을 관리합니다.

예시: 프로젝트 규칙 문서 작성

- 여러 AI 코딩 도구는 프로젝트 폴더의 규칙 문서를 자동으로 참조하는 기능을 제공합니다.

```
# 프로젝트 규칙

## 기술 스택
Backend: FastAPI, PostgreSQL / Frontend: React, TypeScript

## 코딩 컨벤션
함수명은 동사로 시작한다 (예: getUser, createOrder)

## 금지 사항
프로덕션 코드에 콘솔 로그를 남기지 않는다
```

규칙 문서 관리를 위한 3단계 폴더 계층 구조

- 대부분의 AI 도구는 계층화된 MD 설정을 지원하며, 우선순위가 낮은 것부터 높은 것까지 다음과 같이 분류됩니다.
 - 사용자 폴더(전역): 모든 프로젝트에 공통으로 적용할 개인 규칙을 둡니다. (예: 코딩 스타일 취향, 응답 언어 설정, 모든 프로젝트의 공통 규칙)
 - 프로젝트 폴더(루트): 특정 프로젝트에만 적용되는 규칙을 둡니다. 저장소에 함께 커밋되어 팀원과 공유됩니다. (예: 이 프로젝트의 기술 스택, 팀 컨벤션)
 - 특정 작업 폴더(하위): 프로젝트 내 특정 폴더·특정 컴포넌트나 모듈용 전용으로 적용되는 세부 규칙을 둡니다. (예: 프론트엔드 폴더의 UI 컨벤션, 백엔드 폴더의 API 규칙, 로컬 오버라이드)
- 일반적으로 더 좁은 범위(하위 폴더)의 규칙이 상위 규칙 위에 더해지거나 우선 적용되는 방식으로 결합됩니다.

MD 파일의 작동 원리

- AI는 시작 시 자동으로 MD 파일을 검색하고, 그 내용을 컨텍스트(즉, AI의 "기억")로 로드합니다.
- 검색은 재귀적으로 이루어지며, 현재 디렉토리에서 프로젝트 루트 디렉토리까지 거슬러 올라가 발견된 모든 MD 파일이 통합됩니다.
- 이때 현재 디렉토리에 가까운 설정일수록 우선순위가 높습니다.
 - 서브디렉토리의 우선순위가 높은 것은 특정 모듈에는 특별한 규칙이 필요하고, 그것이 프로젝트 전체 규약에 영향을 주지 않도록 할 수 있기 때문입니다.

3층 설정 시스템 예시: 글로벌 설정

- 자주 변경되지 않는 "스타일/선호도"는 글로벌에 두면 각 프로젝트에서 반복해서 작성하는 수고를 덜 수 있습니다.

```
# 내 개발 환경 설정
```

```
## 공통 코딩 규약
```

- 모든 코드에 상세한 주석 포함
- 함수명은 카멜케이스 사용
- JavaScript보다 TypeScript 선호
- 오류 처리는 try-catch 블록 사용

```
## 선호하는 기술 스택
```

- 프론트엔드: React + TypeScript + Tailwind CSS
- 백엔드: Node.js + Express + MongoDB
- 테스트: Jest + React Testing Library

3층 설정 시스템 예시 : 프로젝트 루트 디렉토리 설정

- 현재 디렉토리에서 프로젝트 루트 디렉토리 사이에 있는 규칙 파일입니다.

EC 사이트 프로젝트

프로젝트 개요

이것은 마이크로서비스 아키텍처 기반 EC 사이트로, 사용자 관리, 상품 관리, 주문 처리 등의 주요 기능을 포함합니다.

프로젝트 특정 규약

- API는 RESTful 설계를 따름
- 모든 데이터베이스 작업에는 Prisma ORM 사용
- 프론트엔드 컴포넌트는 반응형 디자인 지원
- 결제 관련 코드에는 추가 보안 검사 필요

디렉토리 구조

- `/src/components/` - 재사용 가능한 React 컴포넌트
- `/src/pages/` - 페이지 레벨 컴포넌트
- `/src/api/` - API 정의
- `/src/utils/` - 유틸리티 함수

3층 설정 시스템 예시 : 컴포넌트 디렉토리 설정

- 로컬 규칙(컴포넌트 디렉토리 등)의 우선순위가 가장 높으며, 공통 규칙의 "예외"를 커버하는데 적합합니다.

```
# 컴포넌트 개발 규약

## 컴포넌트 설계 원칙
- 각 컴포넌트에는 해당하는 .stories.js 파일 필요
- 컴포넌트는 다크 모드 지원
- 모든 props에는 TypeScript 타입 정의 필요
- 컴포넌트에는 단위 테스트 포함

## 스타일 규약
- Tailwind CSS 클래스명 사용
- 인라인 스타일 피하기
- 반응형 디자인 우선
```

규칙문서 관리 예시

- 도구마다 '전역 → 프로젝트 → 하위 작업'이라는 계층 구조 자체는 비슷하게 적용됩니다.
- 예 : Claude vs Cursor

계층	Claude Code	Cursor
사용자(전역)	사용자 홈 폴더의 CLAUDE.md	전역 Rules 설정
프로젝트(루트)	프로젝트 루트의 CLAUDE.md	프로젝트 루트의 규칙 파일 (.cursor/rules)
작업 폴더(하위)	하위 폴더별 CLAUDE.md	폴더별로 범위를 지정한 규칙 파일

☑ 정확한 설정 방법은 각 도구의 최신 공식 문서를 확인하세요

세심하게 설계된 규칙 파일의 효과

- 규칙 파일은 단순한 설정 파일이 아니라 개발 효율성과 협업 경험을 향상시키는 사용자와 AI 어시스턴트 사이의 "계약"입니다.
 - 개발 효율성 향상: 반복적인 설명이 필요 없어지고 AI가 직접 사용자의 요구사항을 이해
 - 코드 품질 보장: 통일된 규약으로 팀 협업이 원활하게 함
 - 학습 비용 감소: 새로운 팀원이 설정을 통해 프로젝트 규약을 빠르게 이해
 - 개인화: AI 어시스턴트가 사용자의 개발 습관에 완전히 적응

MD 문서 관리 시 유의점

- SSOT(단일 진실 공급원) 유지: 같은 정보를 여러 문서에 중복 기술하지 않습니다.
- 컨텍스트 과부하 주의: 문서가 지나치게 길면 핵심 규칙이 오히려 묻힐 수 있어, 핵심 문서와 상세 문서를 분리합니다.
- 주기적 갱신: 코드·정책 변경 시 문서 업데이트를 팀 워크플로우에 정례화합니다.
- 사람의 최종 검토: AI가 초안을 작성한 문서도 반영 전 사람이 검증합니다.

3) 검증 시스템 구축으로 할루시네이션 제거



할루시네이션이란?

- 생성형 AI가 사실이 아니거나 존재하지 않는 정보를 그럴듯하게 만들어내는 현상입니다.
- 코딩 맥락에서는 다음과 같은 형태로 나타납니다.
 - 존재하지 않는 함수, 라이브러리, API를 실제 있는 것처럼 호출
 - 그럴듯하지만 실제로는 틀린 로직이나 설명을 제시
 - 검증되지 않은 내용을 확신에 찬 어조로 전달

AI 코딩에서 흔한 할루시네이션 유형

- 존재하지 않는 패키지나 API 호출: 실제로 배포되지 않은 라이브러리나 함수를 제안
- 오래된 정보 기반 제안: 학습 시점 이후 변경된 문법이나 지원 종료된 기능을 안내
- 근거 없는 성능·동작 주장: 직접 실행·검증하지 않고도 '정상 작동한다'고 서술
- 맥락 오해로 인한 로직 오류: 프로젝트의 실제 요구사항과 다른 방향으로 구현

할루시네이션이 발생하는 이유

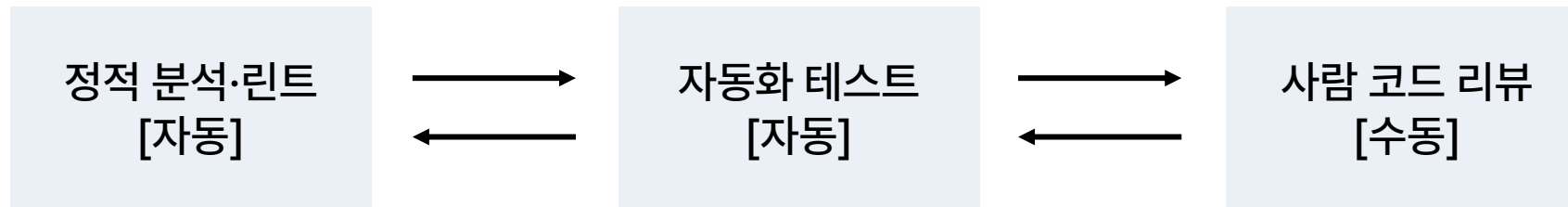
- 통계적 패턴 기반 생성:
 - 사실 검증 절차 없이 '다음에 올 확률이 높은 표현'을 예측해 생성합니다.
- 학습 데이터의 시점 한계:
 - 최신 라이브러리 버전이나 정책 변경이 반영되지 않을 수 있습니다.
- 불충분한 컨텍스트:
 - 프로젝트 정보가 부족하면 AI가 빈틈을 추측으로 채웁니다.

검증 시스템이 왜 필요한가

- AI의 출력은 완성본이 아니라 '검토가 필요한 초안'으로 다루는 것이 안전합니다.
 - 특히 인증, 결제, 개인정보 처리처럼 민감한 영역은 검증 없는 반영이 치명적 위험으로 이어질 수 있습니다.
 - 검증 절차를 체계화하면 AI 활용의 생산성과 신뢰성을 동시에 확보할 수 있습니다.

다층적 검증 체계

- 정적 분석·린트: 문법 오류와 타입 불일치를 즉시 탐지
- 자동화 테스트: 기능이 의도대로 동작하는지 검증
- 사람 코드 리뷰: 자동화가 놓친 맥락과 비즈니스 로직을 판단



자동화된 검증: 정적 분석과 테스트

- 자동 문법 검사: 문법 오류와 타입 불일치를 코드 실행 전에 탐지합니다.
- 자동화 테스트(단위/통합): 기능이 의도한 대로 동작하는지 검증합니다. (TDD 참고)
- CI 파이프라인: 코드 반영 전 검증 단계를 자동 실행해 사람이 보기 전에 1차로 걸러냅니다.

사람의 검증: 코드 리뷰와 도메인 확인

- 자동화 도구가 잡아내지 못하는 비즈니스 로직·맥락 오류는 사람이 판단합니다.
 - 코드 리뷰 체크리스트화: 요구사항 충족 여부, 예외 처리, 보안 취약점을 항목별로 점검
 - 도메인 전문가 확인: 금융, 의료 등 전문 영역은 담당자의 사실 확인이 반드시 필요

사실 기반 검증 (Grounding)

- 공식 문서와 API 레퍼런스에 AI의 답변을 대조하는 습관을 들입니다.
- 출처를 요구하고, 확인 가능한 근거가 있는지 되물어봅니다.
- 중요한 사실은 AI의 단일 응답에 의존하지 않고 다른 경로로 재확인(Cross-check)합니다.

할루시네이션의 해결

- AI 기반 콘텐츠의 개발 과정은 할루시네이션 현상이 일어나지 않도록 검증하는 것이 중요합니다.
 - 특히 정보의 정확성이 중요한 퀴즈 게임이라면 생성된 문제들 중 명백한 오류는 없어 보이지만, 애매한 경우가 있을 수 있습니다.
- 논란의 여지가 있는 주제는 문제 제작 시 교차 검증을 거쳐 명확한 기준이 필요합니다.
 - 논란 있는 정보의 검증을 위해 검증 가이드라인을 마련하고, 이를 MD파일에 영구적으로 저장하는 것이 가장 효과적입니다.

교차 검증 가이드 라인의 필요성

- **정확성이 중요한 콘텐츠는 교차 검증 가이드 라인이 필수입니다.**
- **교차검증 가이드라인이란?**
 - AI의 답변을 외부 신뢰 소스, 전문가 자문, 또는 다른 AI 에이전트와 비교하여 사실관계를 확인하는 과정입니다.
 - 교차 검증 예시
 - 인터넷 검색/전문가 자문: AI의 답변이 의심될 경우, 다수의 신뢰할 수 있는 소스를 통해 사실관계를 검증
 - 자동화 에이전트 사용: Claude Code와 같은 AI 도구를 사용하여 문장 단위 팩트체크 자동화
 - 수학적/논리적 추론 검사: Amazon Bedrock의 자동 추론 검사 기능을 활용하여 결과의 논리적 오류 방지

할루시네이션 점검 체크리스트 예시

- AI 코드를 프로젝트에 반영하기 전, 아래와 같은 체크리스트로 점검하게 합니다.

코드 반영 전 체크리스트

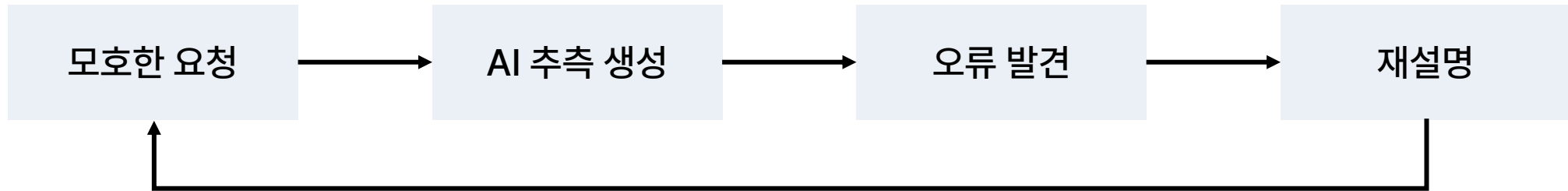
- [] 사용한 모든 함수·라이브러리·API가 실제로 존재하며, 정확한 이름과 사용법으로 호출되었는지 확인한다.
- [] 사용한 문법과 API가 최신 공식 문서 기준과 일치하는지 확인한다.
- [] 지원이 종료되었거나 변경된 기능은 아닌지 점검한다.
- [] 확신할 수 없는 부분은 추측해서 단정하지 않고, "확인이 필요합니다"라고 명시한다.

4) 지속 가능한 AI 개발 관리 프로세스

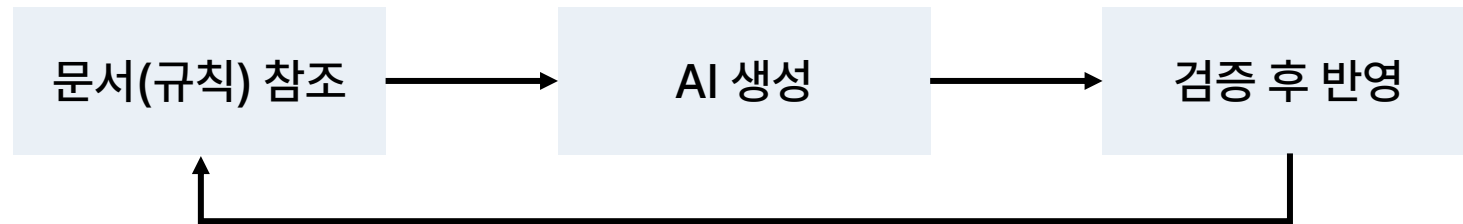


문서와 검증이 만드는 개발 방식의 차이

- 문서와 검증 없이 반복하면 같은 실수를 되풀이하기 쉽습니다.



- 문서와 검증을 결합하면 반복할수록 정확도가 높아집니다.



버전 관리(Git)는 AI 개발의 안전망

- 작은 단위로 커밋해 변경 이력을 추적 가능한 상태로 유지합니다.
- 문제가 발견되면 이전 상태로 되돌릴 수 있는 롤백 지점을 확보합니다.
- 커밋 메시지 자체가 '무엇을, 왜 바꿨는지'를 남기는 또 하나의 문서 역할을 합니다.

체계적인 AI 개발·관리를 위한 원칙 정리

- 문서화: 프로젝트의 목적과 규칙을 MD 문서로 명문화해 컨텍스트를 표준화합니다.
- 검증: 자동화 도구와 사람의 검토를 결합한 다층 방어선을 구축합니다.
- 반복 개선: 검증 결과를 다시 문서에 반영해 다음 작업의 정확도를 높입니다.



THANK YOU!